

# Java Core Deep Understanding Questions [DSA Immersion 2026]

## Java Structure & Execution

1. Why is Java called a platform-independent language?
2. Explain the complete Java execution flow.
3. Difference between JDK, JRE, and JVM?
4. What is bytecode?
5. Why does Java use bytecode?
6. Why is JVM platform dependent?
7. Why is bytecode platform independent?
8. What happens internally when we compile a Java program?
9. What happens internally when we run a Java program?
10. Why is Java called both compiled and interpreted language?

## Class & File Structure

11. Why is class mandatory in Java?
12. Why is Java called a class-based language?
13. Why must file name match public class name?
14. What happens if public class name and file name differ?
15. If class is not public, can file name be anything?
16. Can multiple classes exist in a single Java file?
17. Can multiple public classes exist in one Java file?
18. Why only one public class allowed per file?
19. Difference between class name and file name?
20. Why JVM runs class name instead of file name?

## main() Method Questions

21. Why is `main()` method important?
22. Why is `main()` called entry point of Java?
23. Why is `main` method public?
24. Why is `main` method static?
25. Why is `main` method void?
26. Why does `main` method accept `String[] args`?
27. Can we overload `main` method?
28. Can we override `main` method?
29. Can we make `main` method private?
30. What happens if `main` method is not static?
31. Can Java program execute without `main` method?
32. Can we write `main` method without `String` array?

## Static Keyword Questions

33. What is static keyword?
34. Difference between static and non-static members?
35. Why static methods can be called without object?
36. Why can we access static variables using class name?
37. Why `main` method must be static?
38. Can static method access non-static variables directly?
39. Can constructor be static?
40. Can static methods be overridden?
41. Difference between static binding and dynamic binding?

## Object & Memory Questions

42. What happens internally when object is created?
43. Explain:  

```
Student s = new Student();
```
44. Difference between reference variable and object?
45. Where objects are stored in memory?
46. Difference between Stack memory and Heap memory?
47. What stores local variables?
48. What stores objects?
49. Why stack memory is faster than heap memory?
50. What is garbage collection?

## System.out.println()

51. Explain:  

```
System.out.println();
```
52. What is `System` in Java?
53. What is `out` in Java?
54. What is `println()`?
55. Why can we call `println()` without creating object?
56. Why is `out` static?
57. Which class contains `println()` method?
58. Difference between `print()` and `println()`?
59. What is `PrintStream` class?

## OOP Questions

60. What are the four pillars of OOP?
61. Explain encapsulation with real-world example.
62. Explain inheritance with example.
63. Explain polymorphism with example.
64. Explain abstraction with example.
65. Difference between abstraction and encapsulation?
66. Difference between overloading and overriding?
67. What is runtime polymorphism?
68. What is compile-time polymorphism?
69. Why multiple inheritance not supported in Java?
70. What is IS-A relationship?
71. What is HAS-A relationship?
72. Why Java is not purely object-oriented?

## String Questions

73. Why `String` is immutable in Java?
74. Difference between `==` and `equals()`?
75. What is `String Constant Pool`?
76. Why strings are stored in SCP?
77. Difference between `String`, `StringBuffer` and `StringBuilder`?
78. Why `String` objects are immutable but `StringBuilder` mutable?

## Constructor Questions

79. What is constructor?
80. Why constructor has same name as class?
81. Why constructor has no return type?
82. Can constructor be private?
83. Can constructor be final?
84. Can constructor be static?
85. Difference between constructor and method?
86. What is constructor overloading?
87. What is constructor chaining?

## final Keyword Questions

88. What is final variable?
89. What is final method?
90. What is final class?
91. Why `String` class is final?
92. Can final method be overridden?
93. Can final variable be modified?

## Paradigm & Java Design Questions

94. What is programming paradigm?
95. What are different programming paradigms?
96. Is Java purely object-oriented?
97. Why Java is called multi-paradigm language?
98. Difference between procedural and object-oriented programming?
99. Why Python is called multi-paradigm language?
100. Why understanding paradigms important for placements?

## Placement-Oriented Rapid Fire

101. Why Java does not support pointers directly?
102. Why Java is secure?
103. Why Java is robust?
104. Why Java uses automatic garbage collection?
105. Difference between compiler and interpreter?
106. Difference between JIT and JVM?
107. What is JIT compiler?
108. Why Java is slower than C?
109. Why arrays are fixed in size?
110. Why Java is widely used in enterprise applications?

## Strong Final Discussion Questions

111. Why companies ask output prediction questions?
112. Why dry run practice important?
113. Why logic building important before DSA?
114. Why syntax alone cannot crack interviews?
115. Why understanding execution flow important?
116. Why debugging skills matter in placements?
117. Why optimization important in coding rounds?
118. Why companies prefer problem solvers over syntax learners?
119. Why DSA requires strong Java fundamentals?
120. What makes a student placement-ready?